

**VNUHCM, UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY**



QUESTIONS ABOUT LINKED LISTS
SUBJECT: FUNDAMENTALS OF PROGRAMMING

INSTRUCTORS: Huỳnh Lâm Hải Đăng
Lê Nhật Nam
Class: 25C01

No.	Student ID	Full name
1	25127113	Võ Thiện Trí Nhân

1 Linked List Deletion

1.1 Algorithm

In the linked list, we have nodes and a pointer (whose name is `pHead`) that holds the address of the first node in the list. If we accidentally lose this pointer, we will lose access to the whole list, which causes memory leaks. To delete the list safely, we should follow these steps:

1. Initialize a pointer that also holds the address of the first node.
2. Move `pHead` to the next node.
3. Delete the current node that the temporary pointer is pointing at.
4. Relocate the temporary pointer to the next node.

We continue the steps until the `pHead` is pointing at `nullptr`.

1.2 Code

```
1 struct Node {
2     int data;
3     Node* pNext;
4 };
5
6 struct List {
7     Node* pHead;
8 };
9
10 void deleteList(List& l) { // We need to pass by reference because we want to permanently
11     // change the value of pHead
12     Node* temp = l.pHead; // We initialize a temporary pointer that holds the address of the
13     // first node
14
15     while (l.pHead != nullptr) {
16         l.pHead = l.pHead->pNext; // Move the pHead to the next node
17         delete temp; // Delete the previous node
18         temp = l.pHead; // Move the temp pointer to the next node
19     }
20 }
```

Listing 1: Linked List Deletion

2 Pass-by-reference Pointer

2.1 How does it work?

When we pass by value, the function will initialize a copy of the passed variable and use that copy to operate in the scope. The value of the passed variable will not be modified after the function stops.

When we pass by reference, the function will work directly with the passed variable. If there is a change in the value of the passed variable, it will be a permanent modification. There is only one available syntax.

```
1 void AddHead(<data type>*& <name>);
```

Therefore, `void AddHead(Node*& pHead, int data);` is valid and the compiler compiles smoothly. However, `void AddHead(Node&* pHead, int data);` is completely invalid which means the compiler will shout if we use this syntax.

2.2 Test

We have this code snippet

```
1 #include <iostream>
2 using namespace std;
3
4 struct Node {
5     int data;
6     Node* next;
7 };
8
9 struct List {
10     Node* head;
11 };
12
13 void Init(List& l) {
14     l.head = nullptr;
15 }
16
17 Node* CreateNode(int x) {
18     Node* p = new Node;
19     if (p == nullptr) return nullptr;
20     p->data = x;
21     p->next = nullptr;
22     return p;
23 }
24
25 // Focus only on this function
26 void AddHead(Node*& head, int x) {
27     Node* p = CreateNode(x);
28     if (head == nullptr) {
29         head = p;
30     } else {
31         p->next = head;
32         head = p;
33     }
34 }
35
36 void AddTail(List& l, int x) {
37     Node* p = CreateNode(x);
38     if (l.head == nullptr) {
39         l.head = p;
40     } else {
41         Node* temp = l.head;
```

```

42     while (temp->next != nullptr) {
43         temp = temp->next;
44     }
45     temp->next = p;
46 }
47 }
48
49 void PrintList(List l) {
50     Node* temp = l.head;
51     while (temp != nullptr) {
52         cout << temp->data << " -> ";
53         temp = temp->next;
54     }
55     cout << "NULL" << endl;
56 }
57
58 int main() {
59     List l;
60     Init(l);
61
62     AddTail(l, 2);
63     AddTail(l, 3);
64     AddTail(l, 4);
65     PrintList(l);
66
67     AddHead(l.head, 1);
68     PrintList(l);
69
70     return 0;
71 }

```

Result after compilation:

```

1 2 -> 3 -> 4 -> NULL
2 1 -> 2 -> 3 -> 4 -> NULL

```

If we modify `Node*& head` to `Node&* head`, the compiler will shout.

Result after compilation:

```

1 Main.cpp:25:21: error: cannot declare pointer to 'struct Node&'
2 25 | void AddHead(Node&* head, int x) {
3     |                  ^~~~

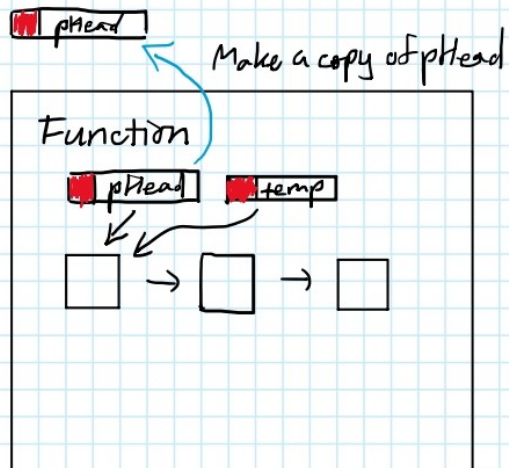
```

2.3 Visualization

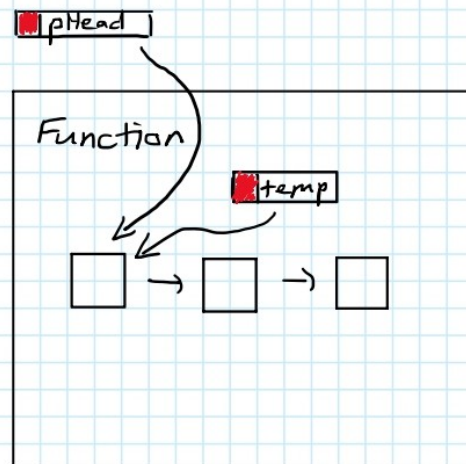
Visualization

Sunday, December 28, 2025 11:24 AM

```
void Addhead(Node* pHead, int data);
```



```
void Addhead(Node*& pHead, int data);
```



```
void Addhead(Node& pHead, int data);  
INVALID!!!
```

Hình 1: Visualization on Node*, Node*& and Node&*